DBMS LAB ASSIGNMENT

Labs 7 to 10

Joins | Grouping and Ordering | Procedures | Triggers

Name:	priyanshu
Roll No / Reg No:	_____
Course:	BCA, R.V. University
Database used:	MySQL / MariaDB 10.11

Lab 7	Implement different types of Joins
Lab 8	Implement Grouping and Ordering (GROUP BY, HAVING, ORDER BY)
Lab 9	Implement Procedures (basic, Insert, Update, Delete)
Lab 10	Implement Triggers (validation before/after insert, update, delete)

LAB 7: Implement different types of Joins

Aim: To implement the different types of joins in SQL: Inner Join, Outer Joins (Left, Right, Full), Natural Join, plus Cross and Self Join.

1. Create database and tables

INPUT (SQL)

```
DROP DATABASE IF EXISTS lab7;
CREATE DATABASE lab7;
USE lab7;
CREATE TABLE departments (
  dept_id   INT PRIMARY KEY,
  dept_name VARCHAR(30)
);
CREATE TABLE employees (
  emp_id   INT PRIMARY KEY,
  emp_name VARCHAR(30),
  dept_id  INT,
  salary   INT
);
SHOW TABLES;
```

OUTPUT

```
+----------------+
| Tables_in_lab7 |
+----------------+
| departments    |
| employees      |
+----------------+
```

2. Insert sample data (Eve has no department; Legal has no employees)

INPUT (SQL)

```
INSERT INTO departments VALUES
 (1,'IT'),(2,'HR'),(3,'Finance'),(4,'Legal');
INSERT INTO employees VALUES
 (101,'Asha',1,60000),(102,'Ravi',1,55000),(103,'Meera',2,45000),
 (104,'Karan',3,70000),(105,'Eve',NULL,40000);
SELECT * FROM departments;
SELECT * FROM employees;
```

OUTPUT

```
+---------+-----------+
| dept_id | dept_name |
+---------+-----------+
|       1 | IT        |
|       2 | HR        |
|       3 | Finance   |
|       4 | Legal     |
+---------+-----------+
+--------+----------+---------+--------+
| emp_id | emp_name | dept_id | salary |
+--------+----------+---------+--------+
|    101 | Asha     |       1 |  60000 |
|    102 | Ravi     |       1 |  55000 |
|    103 | Meera    |       2 |  45000 |
|    104 | Karan    |       3 |  70000 |
|    105 | Eve      |    NULL |  40000 |
+--------+----------+---------+--------+
```

3. INNER JOIN - only rows that match in both tables

INPUT (SQL)

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

OUTPUT

```
+--------+----------+-----------+
| emp_id | emp_name | dept_name |
+--------+----------+-----------+
|    101 | Asha     | IT        |
|    102 | Ravi     | IT        |
|    103 | Meera    | HR        |
|    104 | Karan    | Finance   |
+--------+----------+-----------+
```

4. LEFT OUTER JOIN - all employees, even without a department

INPUT (SQL)

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

OUTPUT

```
+--------+----------+-----------+
| emp_id | emp_name | dept_name |
+--------+----------+-----------+
|    101 | Asha     | IT        |
|    102 | Ravi     | IT        |
|    103 | Meera    | HR        |
|    104 | Karan    | Finance   |
|    105 | Eve      | NULL      |
+--------+----------+-----------+
```

5. RIGHT OUTER JOIN - all departments, even without employees

INPUT (SQL)

```
SELECT e.emp_name, d.dept_id, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

OUTPUT

```
+----------+---------+-----------+
| emp_name | dept_id | dept_name |
+----------+---------+-----------+
| Asha     |       1 | IT        |
| Ravi     |       1 | IT        |
| Meera    |       2 | HR        |
| Karan    |       3 | Finance   |
| NULL     |       4 | Legal     |
+----------+---------+-----------+
```

6. FULL OUTER JOIN - emulated using UNION (MySQL has no FULL JOIN keyword)

INPUT (SQL)

```
SELECT e.emp_name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

OUTPUT

```
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Asha     | IT        |
| Ravi     | IT        |
| Meera    | HR        |
| Karan    | Finance   |
| Eve      | NULL      |
| NULL     | Legal     |
+----------+-----------+
```

7. NATURAL JOIN - joins automatically on the common column (dept_id)

INPUT (SQL)

```
SELECT emp_id, emp_name, dept_name
FROM employees
NATURAL JOIN departments;
```

OUTPUT

```
+--------+----------+-----------+
| emp_id | emp_name | dept_name |
+--------+----------+-----------+
|    101 | Asha     | IT        |
|    102 | Ravi     | IT        |
|    103 | Meera    | HR        |
|    104 | Karan    | Finance   |
+--------+----------+-----------+
```

8. CROSS JOIN - every employee paired with every department (first 6 rows)

INPUT (SQL)

```
SELECT e.emp_name, d.dept_name
FROM employees e
CROSS JOIN departments d
ORDER BY e.emp_id, d.dept_id
LIMIT 6;
```

OUTPUT

```
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Asha     | IT        |
| Asha     | HR        |
| Asha     | Finance   |
| Asha     | Legal     |
| Ravi     | IT        |
| Ravi     | HR        |
+----------+-----------+
```

9. SELF JOIN - employees who earn more than Ravi

INPUT (SQL)

```
SELECT a.emp_name AS employee, b.emp_name AS compared_with, a.salary
FROM employees a
JOIN employees b ON a.salary > b.salary AND b.emp_name = 'Ravi';
```

OUTPUT

```
+----------+---------------+--------+
| employee | compared_with | salary |
+----------+---------------+--------+
| Asha     | Ravi          |  60000 |
| Karan    | Ravi          |  70000 |
+----------+---------------+--------+
```

LAB 8: Implement Grouping and Ordering (GROUP BY, HAVING, ORDER BY)

Aim: To study and implement the GROUP BY and HAVING clauses and the ORDER BY clause.

1. Create database, table and insert data

INPUT (SQL)

```
DROP DATABASE IF EXISTS lab8;
CREATE DATABASE lab8;
USE lab8;
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  name   VARCHAR(30),
  dept   VARCHAR(20),
  city   VARCHAR(20),
  salary INT
);
INSERT INTO employees VALUES
 (1,'Asha','IT','Bangalore',60000),
 (2,'Ravi','IT','Udaipur',55000),
 (3,'Meera','HR','Bangalore',45000),
 (4,'Karan','Finance','Delhi',70000),
 (5,'Neha','IT','Bangalore',65000),
 (6,'Imran','HR','Delhi',48000),
 (7,'Priya','Finance','Bangalore',72000);
SELECT * FROM employees;
```

OUTPUT

```
+--------+-------+---------+-----------+--------+
| emp_id | name  | dept    | city      | salary |
+--------+-------+---------+-----------+--------+
|      1 | Asha  | IT      | Bangalore |  60000 |
|      2 | Ravi  | IT      | Udaipur   |  55000 |
|      3 | Meera | HR      | Bangalore |  45000 |
|      4 | Karan | Finance | Delhi     |  70000 |
|      5 | Neha  | IT      | Bangalore |  65000 |
|      6 | Imran | HR      | Delhi     |  48000 |
|      7 | Priya | Finance | Bangalore |  72000 |
+--------+-------+---------+-----------+--------+
```

2. GROUP BY - count and average salary per department

INPUT (SQL)

```
SELECT dept, COUNT(*) AS total_emp, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept;
```

OUTPUT

```
+---------+-----------+------------+
| dept    | total_emp | avg_salary |
+---------+-----------+------------+
| Finance |         2 | 71000.0000 |
| HR      |         2 | 46500.0000 |
| IT      |         3 | 60000.0000 |
+---------+-----------+------------+
```

3. GROUP BY with SUM, MIN and MAX

INPUT (SQL)

```
SELECT dept, SUM(salary) AS total_salary,
       MIN(salary) AS lowest, MAX(salary) AS highest
FROM employees
GROUP BY dept;
```

OUTPUT

```
+---------+--------------+--------+---------+
| dept    | total_salary | lowest | highest |
+---------+--------------+--------+---------+
| Finance |       142000 |  70000 |   72000 |
| HR      |        93000 |  45000 |   48000 |
| IT      |       180000 |  55000 |   65000 |
+---------+--------------+--------+---------+
```

4. GROUP BY on multiple columns

INPUT (SQL)

```
SELECT dept, city, COUNT(*) AS total_emp
FROM employees
GROUP BY dept, city;
```

OUTPUT

```
+---------+-----------+-----------+
| dept    | city      | total_emp |
+---------+-----------+-----------+
| Finance | Bangalore |         1 |
| Finance | Delhi     |         1 |
| HR      | Bangalore |         1 |
| HR      | Delhi     |         1 |
| IT      | Bangalore |         2 |
| IT      | Udaipur   |         1 |
+---------+-----------+-----------+
```

5. HAVING - keep only groups with average salary above 55000

INPUT (SQL)

```
SELECT dept, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept
HAVING AVG(salary) > 55000;
```

OUTPUT

```
+---------+------------+
| dept    | avg_salary |
+---------+------------+
| Finance | 71000.0000 |
| IT      | 60000.0000 |
+---------+------------+
```

6. WHERE + GROUP BY + HAVING together (Bangalore only, departments with 2+ people)

INPUT (SQL)

```
SELECT dept, COUNT(*) AS total_emp
FROM employees
WHERE city = 'Bangalore'
GROUP BY dept
HAVING COUNT(*) >= 2;
```

OUTPUT

```
+------+-----------+
| dept | total_emp |
+------+-----------+
| IT   |         2 |
+------+-----------+
```

7. ORDER BY ascending (default)

INPUT (SQL)

```
SELECT name, dept, salary
FROM employees
ORDER BY salary;
```

OUTPUT

```
+-------+---------+--------+
| name  | dept    | salary |
+-------+---------+--------+
| Meera | HR      |  45000 |
| Imran | HR      |  48000 |
| Ravi  | IT      |  55000 |
| Asha  | IT      |  60000 |
| Neha  | IT      |  65000 |
| Karan | Finance |  70000 |
| Priya | Finance |  72000 |
+-------+---------+--------+
```

8. ORDER BY descending

INPUT (SQL)

```
SELECT name, dept, salary
FROM employees
ORDER BY salary DESC;
```

OUTPUT

```
+-------+---------+--------+
| name  | dept    | salary |
+-------+---------+--------+
| Priya | Finance |  72000 |
| Karan | Finance |  70000 |
| Neha  | IT      |  65000 |
| Asha  | IT      |  60000 |
| Ravi  | IT      |  55000 |
| Imran | HR      |  48000 |
| Meera | HR      |  45000 |
+-------+---------+--------+
```

9. ORDER BY multiple columns (dept A-Z, then salary high to low)

INPUT (SQL)

```
SELECT name, dept, salary
FROM employees
ORDER BY dept ASC, salary DESC;
```

OUTPUT

```
+-------+---------+--------+
| name  | dept    | salary |
+-------+---------+--------+
| Priya | Finance |  72000 |
| Karan | Finance |  70000 |
| Imran | HR      |  48000 |
| Meera | HR      |  45000 |
| Neha  | IT      |  65000 |
| Asha  | IT      |  60000 |
| Ravi  | IT      |  55000 |
+-------+---------+--------+
```

10. GROUP BY + HAVING + ORDER BY combined

INPUT (SQL)

```
SELECT dept, COUNT(*) AS total_emp, SUM(salary) AS total_salary
FROM employees
GROUP BY dept
HAVING SUM(salary) > 100000
ORDER BY total_salary DESC;
```

OUTPUT

```
+---------+-----------+--------------+
| dept    | total_emp | total_salary |
+---------+-----------+--------------+
| IT      |         3 |       180000 |
| Finance |         2 |       142000 |
+---------+-----------+--------------+
```

LAB 9: Implement Procedures (basic, Insert, Update, Delete)

Aim: To implement stored procedures: a basic procedure and procedures for Insert, Update and Delete.

1. Create database, table and insert data

INPUT (SQL)

```
DROP DATABASE IF EXISTS lab9;
CREATE DATABASE lab9;
USE lab9;
CREATE TABLE students (
  roll_no INT PRIMARY KEY,
  name    VARCHAR(30),
  course  VARCHAR(20),
  marks   INT
);
INSERT INTO students VALUES
 (1,'priyanshu','BCA',82),(2,'Aman','BCA',74),(3,'Riya','BCA',91);
SELECT * FROM students;
```

OUTPUT

```
+---------+--------+--------+-------+
| roll_no | name   | course | marks |
+---------+--------+--------+-------+
|       1 | priyanshu | BCA    |    82 |
|       2 | Aman   | BCA    |    74 |
|       3 | Riya   | BCA    |    91 |
+---------+--------+--------+-------+
```

2. Basic procedure (no parameters) - display all students

INPUT (SQL)

```
DELIMITER //
CREATE PROCEDURE sp_show_students()
BEGIN
  SELECT * FROM students;
END //
DELIMITER ;
CALL sp_show_students();
```

OUTPUT

```
+---------+--------+--------+-------+
| roll_no | name   | course | marks |
+---------+--------+--------+-------+
|       1 | priyanshu | BCA    |    82 |
|       2 | Aman   | BCA    |    74 |
|       3 | Riya   | BCA    |    91 |
+---------+--------+--------+-------+
```

3. Procedure with IN and OUT parameters - get marks of a student

INPUT (SQL)

```
DELIMITER //
CREATE PROCEDURE sp_get_marks(IN p_roll INT, OUT p_marks INT)
BEGIN
  SELECT marks INTO p_marks FROM students WHERE roll_no = p_roll;
END //
DELIMITER ;
CALL sp_get_marks(3, @m);
SELECT @m AS marks_of_roll_3;
```

OUTPUT

```
+-----------------+
| marks_of_roll_3 |
+-----------------+
|              91 |
+-----------------+
```

4. Procedure for INSERT

INPUT (SQL)

```
DELIMITER //
CREATE PROCEDURE sp_insert_student(
  IN p_roll INT, IN p_name VARCHAR(30), IN p_course VARCHAR(20), IN p_marks INT)
BEGIN
  INSERT INTO students VALUES (p_roll, p_name, p_course, p_marks);
END //
DELIMITER ;
CALL sp_insert_student(4, 'Neha', 'BCA', 68);
CALL sp_insert_student(5, 'Karan', 'BCA', 77);
SELECT * FROM students;
```

OUTPUT

```
+---------+--------+--------+-------+
| roll_no | name   | course | marks |
+---------+--------+--------+-------+
|       1 | priyanshu | BCA    |    82 |
|       2 | Aman   | BCA    |    74 |
|       3 | Riya   | BCA    |    91 |
|       4 | Neha   | BCA    |    68 |
|       5 | Karan  | BCA    |    77 |
+---------+--------+--------+-------+
```

5. Procedure for UPDATE

INPUT (SQL)

```
DELIMITER //
CREATE PROCEDURE sp_update_marks(IN p_roll INT, IN p_marks INT)
BEGIN
  UPDATE students SET marks = p_marks WHERE roll_no = p_roll;
END //
DELIMITER ;
CALL sp_update_marks(2, 80);
SELECT * FROM students;
```

OUTPUT

```
+---------+--------+--------+-------+
| roll_no | name   | course | marks |
+---------+--------+--------+-------+
|       1 | priyanshu | BCA    |    82 |
|       2 | Aman   | BCA    |    80 |
|       3 | Riya   | BCA    |    91 |
|       4 | Neha   | BCA    |    68 |
|       5 | Karan  | BCA    |    77 |
+---------+--------+--------+-------+
```

6. Procedure for DELETE

INPUT (SQL)

```
DELIMITER //
CREATE PROCEDURE sp_delete_student(IN p_roll INT)
BEGIN
  DELETE FROM students WHERE roll_no = p_roll;
END //
DELIMITER ;
CALL sp_delete_student(5);
SELECT * FROM students;
```

OUTPUT

```
+---------+--------+--------+-------+
| roll_no | name   | course | marks |
+---------+--------+--------+-------+
|       1 | priyanshu | BCA    |    82 |
|       2 | Aman   | BCA    |    80 |
|       3 | Riya   | BCA    |    91 |
|       4 | Neha   | BCA    |    68 |
+---------+--------+--------+-------+
```

7. Procedure with IF/ELSE - grade of a student

INPUT (SQL)

```
DELIMITER //
CREATE PROCEDURE sp_grade(IN p_roll INT)
BEGIN
  DECLARE m INT;
  SELECT marks INTO m FROM students WHERE roll_no = p_roll;
  IF m >= 90 THEN SELECT p_roll AS roll_no, m AS marks, 'A+' AS grade;
  ELSEIF m >= 75 THEN SELECT p_roll AS roll_no, m AS marks, 'A' AS grade;
  ELSEIF m >= 60 THEN SELECT p_roll AS roll_no, m AS marks, 'B' AS grade;
  ELSE SELECT p_roll AS roll_no, m AS marks, 'C' AS grade;
  END IF;
END //
DELIMITER ;
CALL sp_grade(3);
CALL sp_grade(2);
```

OUTPUT

```
+---------+-------+-------+
| roll_no | marks | grade |
+---------+-------+-------+
|       3 |    91 | A+    |
+---------+-------+-------+
+---------+-------+-------+
| roll_no | marks | grade |
+---------+-------+-------+
|       2 |    80 | A     |
+---------+-------+-------+
```

8. List the stored procedures created

INPUT (SQL)

```
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = 'lab9'
ORDER BY routine_name;
```

OUTPUT

```
+--------------------+--------------+
| routine_name       | routine_type |
+--------------------+--------------+
| sp_delete_student  | PROCEDURE    |
| sp_get_marks       | PROCEDURE    |
| sp_grade           | PROCEDURE    |
| sp_insert_student  | PROCEDURE    |
| sp_show_students   | PROCEDURE    |
| sp_update_marks    | PROCEDURE    |
+--------------------+--------------+
```

LAB 10: Implement Triggers (validation before/after insert, update, delete)

Aim: To implement triggers that validate data before and after Insert, Update and Delete.

1. Create database, tables and insert data

INPUT (SQL)

```
DROP DATABASE IF EXISTS lab10;
CREATE DATABASE lab10;
USE lab10;
CREATE TABLE students (
  roll_no INT PRIMARY KEY,
  name    VARCHAR(30),
  marks   INT
);
CREATE TABLE audit_log (
  log_id     INT AUTO_INCREMENT PRIMARY KEY,
  action     VARCHAR(30),
  roll_no    INT,
  old_marks  INT,
  new_marks  INT
);
INSERT INTO students VALUES (1,'priyanshu',82),(2,'Aman',74);
SELECT * FROM students;
```

OUTPUT

```
+---------+---------+-------+
| roll_no | name    | marks |
+---------+---------+-------+
|       1 | priyanshu |    82 |
|       2 | Aman    |    74 |
+---------+---------+-------+
```

2. BEFORE INSERT trigger - validate marks are between 0 and 100

INPUT (SQL)

```
DELIMITER //
CREATE TRIGGER trg_before_insert
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
  IF NEW.marks < 0 OR NEW.marks > 100 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Invalid marks: must be between 0 and 100';
  END IF;
END //
DELIMITER ;
INSERT INTO students VALUES (3,'Riya',150);
```

OUTPUT

```
ERROR 1644 (45000): Invalid marks: must be between 0 and 100
```

3. Valid insert still works

INPUT (SQL)

```
INSERT INTO students VALUES (3,'Riya',91);
SELECT * FROM students;
```

OUTPUT

```
+---------+---------+-------+
| roll_no | name    | marks |
+---------+---------+-------+
|       1 | priyanshu |    82 |
|       2 | Aman    |    74 |
|       3 | Riya    |    91 |
+---------+---------+-------+
```

4. AFTER INSERT trigger - record the action in audit_log

INPUT (SQL)

```
DELIMITER //
CREATE TRIGGER trg_after_insert
AFTER INSERT ON students
FOR EACH ROW
BEGIN
  INSERT INTO audit_log(action, roll_no, old_marks, new_marks)
  VALUES ('INSERT', NEW.roll_no, NULL, NEW.marks);
END //
DELIMITER ;
INSERT INTO students VALUES (4,'Neha',68);
SELECT * FROM audit_log;
```

OUTPUT

```
+--------+--------+---------+-----------+-----------+
| log_id | action | roll_no | old_marks | new_marks |
+--------+--------+---------+-----------+-----------+
|      1 | INSERT |       4 |      NULL |        68 |
+--------+--------+---------+-----------+-----------+
```

5. BEFORE UPDATE trigger - reject marks outside 0 to 100

INPUT (SQL)

```
DELIMITER //
CREATE TRIGGER trg_before_update
BEFORE UPDATE ON students
FOR EACH ROW
BEGIN
  IF NEW.marks < 0 OR NEW.marks > 100 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Update rejected: marks must be between 0 and 100';
  END IF;
END //
DELIMITER ;
UPDATE students SET marks = 120 WHERE roll_no = 2;
```

OUTPUT

```
ERROR 1644 (45000): Update rejected: marks must be between 0 and 100
```

6. AFTER UPDATE trigger - log old and new marks

INPUT (SQL)

```
DELIMITER //
CREATE TRIGGER trg_after_update
AFTER UPDATE ON students
FOR EACH ROW
BEGIN
  INSERT INTO audit_log(action, roll_no, old_marks, new_marks)
  VALUES ('UPDATE', NEW.roll_no, OLD.marks, NEW.marks);
END //
DELIMITER ;
UPDATE students SET marks = 80 WHERE roll_no = 2;
SELECT * FROM students;
SELECT * FROM audit_log;
```

OUTPUT

```
+---------+---------+-------+
| roll_no | name    | marks |
+---------+---------+-------+
|       1 | priyanshu |    82 |
|       2 | Aman    |    80 |
|       3 | Riya    |    91 |
|       4 | Neha    |    68 |
+---------+---------+-------+
+--------+--------+---------+-----------+-----------+
| log_id | action | roll_no | old_marks | new_marks |
+--------+--------+---------+-----------+-----------+
|      1 | INSERT |       4 |      NULL |        68 |
|      2 | UPDATE |       2 |        74 |        80 |
+--------+--------+---------+-----------+-----------+
```

7. BEFORE DELETE trigger - toppers (marks >= 90) cannot be deleted

INPUT (SQL)

```
DELIMITER //
CREATE TRIGGER trg_before_delete
BEFORE DELETE ON students
FOR EACH ROW
BEGIN
  IF OLD.marks >= 90 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Delete rejected: topper records cannot be deleted';
  END IF;
END //
DELIMITER ;
DELETE FROM students WHERE roll_no = 3;
```

OUTPUT

```
ERROR 1644 (45000): Delete rejected: topper records cannot be deleted
```

8. AFTER DELETE trigger - log the deleted record

INPUT (SQL)

```
DELIMITER //
CREATE TRIGGER trg_after_delete
AFTER DELETE ON students
FOR EACH ROW
BEGIN
  INSERT INTO audit_log(action, roll_no, old_marks, new_marks)
  VALUES ('DELETE', OLD.roll_no, OLD.marks, NULL);
END //
DELIMITER ;
DELETE FROM students WHERE roll_no = 4;
SELECT * FROM students;
SELECT * FROM audit_log;
```

OUTPUT

```
+---------+---------+-------+
| roll_no | name    | marks |
+---------+---------+-------+
|       1 | priyanshu |    82 |
|       2 | Aman    |    80 |
|       3 | Riya    |    91 |
+---------+---------+-------+
+--------+--------+---------+-----------+-----------+
| log_id | action | roll_no | old_marks | new_marks |
+--------+--------+---------+-----------+-----------+
|      1 | INSERT |       4 |      NULL |        68 |
|      2 | UPDATE |       2 |        74 |        80 |
|      3 | DELETE |       4 |        68 |      NULL |
+--------+--------+---------+-----------+-----------+
```

9. List the triggers created

INPUT (SQL)

```
SELECT trigger_name, action_timing, event_manipulation, event_object_table
FROM information_schema.triggers
WHERE trigger_schema = 'lab10'
ORDER BY trigger_name;
```

OUTPUT

```
+-------------------+---------------+--------------------+--------------------+
| trigger_name      | action_timing | event_manipulation | event_object_table |
+-------------------+---------------+--------------------+--------------------+
| trg_after_delete  | AFTER         | DELETE             | students           |
| trg_after_insert  | AFTER         | INSERT             | students           |
| trg_after_update  | AFTER         | UPDATE             | students           |
| trg_before_delete | BEFORE        | DELETE             | students           |
| trg_before_insert | BEFORE        | INSERT             | students           |
| trg_before_update | BEFORE        | UPDATE             | students           |
+-------------------+---------------+--------------------+--------------------+
```